

elements of the first $N/2$ and second $N/2$ at the very end, then all missing communication paths are restored by traveling through the first $N/2$ elements.

The third line in the following code is an idiom that sets n to the next larger power of 2 greater or equal to m , a miniature masterpiece due to S.E. Anderson [2]. If you look closely, you'll see that it is itself a sort of butterfly, but now on bits!

```
void hashall(VecUInt &arr) {
```

hashall.h

Replace the array `arr` by a same-sized hash, all of whose bits depend on all of the bits in `arr`. Uses `psdes` for the mutual hash of two 32-bit words.

```
    Int m=arr.size(), n=m-1;
    n|=n>>1; n|=n>>2; n|=n>>4; n|=n>>8; n|=n>>16; n++;
    Incredibly, n is now the next power of 2  $\geq m$ .
    Int nb=n,nb2=n>>1,j,jb;
    if (n<2) throw("size must be > 1");
    while (nb > 1) {
        for (jb=0;jb<n-nb+1;jb+=nb)
            for (j=0;j<nb2;j++)
                if (jb+j+nb2 < m) psdes(arr[jb+j],arr[jb+j+nb2]);
        nb = nb2;
        nb2 >>= 1;
    }
    nb2 = n>>1;
    if (m != n) for (j=nb2;j<m;j++) psdes(arr[j],arr[j-nb2]);
    Final mix needed only if m is not a power of 2.
```

```
}
```

CITED REFERENCES AND FURTHER READING:

- Meyer, C.H. and Matyas, S.M. 1982, *Cryptography: A New Dimension in Computer Data Security* (New York: Wiley).[1]
- Zonst, A.E. 2000, *Understanding the FFT*, 2nd revised ed. (Titusville, FL: Citrus Press).
- Anderson, S.E. 2005, "Bit Twiddling Hacks," 2007+ at <http://graphics.stanford.edu/~seander/bithacks.html>. [2]
- Data Encryption Standard*, 1977 January 15, Federal Information Processing Standards Publication, number 46 (Washington: U.S. Department of Commerce, National Bureau of Standards).
- Guidelines for Implementing and Using the NBS Data Encryption Standard*, 1981 April 1, Federal Information Processing Standards Publication, number 74 (Washington: U.S. Department of Commerce, National Bureau of Standards).

7.3 Deviates from Other Distributions

In §7.1 we learned how to generate random deviates with a uniform probability between 0 and 1, denoted $U(0, 1)$. The probability of generating a number between x and $x + dx$ is

$$p(x)dx = \begin{cases} dx & 0 \leq x < 1 \\ 0 & \text{otherwise} \end{cases} \quad (7.3.1)$$

and we write

$$x \sim U(0, 1) \quad (7.3.2)$$

As in §6.14, the symbol $\sim$ can be read as "is drawn from the distribution."

In this section, we learn how to generate random deviates drawn from other probability distributions, including all of those discussed in §6.14. Discussion of specific distributions is interleaved with the discussion of the general methods used.

7.3.1 Exponential Deviates

Suppose that we generate a uniform deviate x and then take some prescribed function of it, $y(x)$. The probability distribution of y , denoted $p(y)dy$, is determined by the fundamental transformation law of probabilities, which is simply

$$|p(y)dy| = |p(x)dx| \quad (7.3.3)$$

or

$$p(y) = p(x) \left| \frac{dx}{dy} \right| \quad (7.3.4)$$

As an example, take

$$y(x) = -\ln(x) \quad (7.3.5)$$

with $x \sim U(0, 1)$. Then

$$p(y)dy = \left| \frac{dx}{dy} \right| dy = e^{-y} dy \quad (7.3.6)$$

which is the exponential distribution with unit mean, Exponential(1), discussed in §6.14.5. This distribution occurs frequently in real life, usually as the distribution of waiting times between independent Poisson-random events, for example the radioactive decay of nuclei. You can also easily see (from 7.3.6) that the quantity y/β has the probability distribution $\beta e^{-\beta y}$, so

$$y/\beta \sim \text{Exponential}(\beta) \quad (7.3.7)$$

We can thus generate exponential deviates at a cost of about one uniform deviate, plus a logarithm, per call.

```
deviates.h struct Expondev : Ran {
Structure for exponential deviates.
    Doub beta;
    Expondev(Doub bbeta, Ullong i) : Ran(i), beta(bbeta) {}
    Constructor arguments are  $\beta$  and a random sequence seed.
    Doub dev() {
    Return an exponential deviate.
        Doub u;
        do u = doub(); while (u == 0.);
        return -log(u)/beta;
    }
};
```

Our convention here and in the rest of this section is to derive the class for each kind of deviate from the uniform generator class `Ran`. We use the constructor to set the distribution's parameters and set the initial seed for the generator. We then provide a method `dev()` that returns a random deviate from the distribution.

7.3.2 Transformation Method in General

Let's see what is involved in using the above *transformation method* to generate some arbitrary desired distribution of y 's, say one with $p(y) = f(y)$ for some positive function f whose integral is 1. According to (7.3.4), we need to solve the differential equation

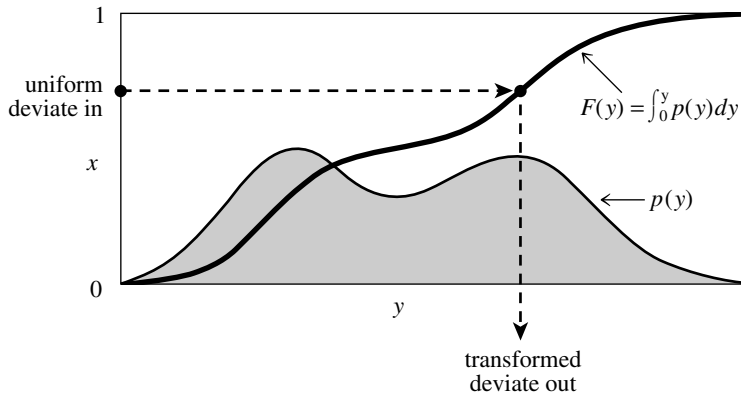


Figure 7.3.1. Transformation method for generating a random deviate y from a known probability distribution $p(y)$. The indefinite integral of $p(y)$ must be known and invertible. A uniform deviate x is chosen between 0 and 1. Its corresponding y on the definite-integral curve is the desired deviate.

$$\frac{dx}{dy} = f(y) \quad (7.3.8)$$

But the solution of this is just $x = F(y)$, where $F(y)$ is the indefinite integral of $f(y)$. The desired transformation that takes a uniform deviate into one distributed as $f(y)$ is therefore

$$y(x) = F^{-1}(x) \quad (7.3.9)$$

where F^{-1} is the inverse function to F . Whether (7.3.9) is feasible to implement depends on whether the *inverse function of the integral of $f(y)$* is itself feasible to compute, either analytically or numerically. Sometimes it is, and sometimes it isn't.

Incidentally, (7.3.9) has an immediate geometric interpretation: Since $F(y)$ is the area under the probability curve to the left of y , (7.3.9) is just the prescription: Choose a uniform random x , then find the value y that has that fraction x of probability area to its left, and return the value y . (See Figure 7.3.1.)

7.3.3 Logistic Deviates

Deviates from the logistic distribution, as discussed in §6.14.4, are readily generated by the transformation method, using equation (6.14.15). The cost is again dominated by one uniform deviate, and a logarithm, per logistic deviate.

```

struct Logisticdev : Ran {
Structure for logistic deviates.
    Doub mu,sig;
    Logisticdev(Doub mmu, Doub ssig, Ullong i) : Ran(i), mu(mmu), sig(ssig) {}
    Constructor arguments are  $\mu$ ,  $\sigma$ , and a random sequence seed.
    Doub dev() {
    Return a logistic deviate.
        Doub u;
        do u = doub(); while (u*(1.-u) == 0.);
        return mu + 0.551328895421792050*sig*log(u/(1.-u));
    }
};

```

deviates.h

7.3.4 Normal Deviates by Transformation (Box-Muller)

Transformation methods generalize to more than one dimension. If $x_1, x_2, \dots$ are random deviates with a *joint* probability distribution $p(x_1, x_2, \dots)dx_1dx_2\dots$, and if $y_1, y_2, \dots$ are each functions of all the x 's (same number of y 's as x 's), then the joint probability distribution of the y 's is

$$p(y_1, y_2, \dots)dy_1dy_2\dots = p(x_1, x_2, \dots) \left| \frac{\partial(x_1, x_2, \dots)}{\partial(y_1, y_2, \dots)} \right| dy_1dy_2\dots \quad (7.3.10)$$

where $|\partial(\quad)/\partial(\quad)|$ is the Jacobian determinant of the x 's with respect to the y 's (or the reciprocal of the Jacobian determinant of the y 's with respect to the x 's).

An important historical example of the use of (7.3.10) is the *Box-Muller* method for generating random deviates with a normal (Gaussian) distribution (§6.14.1):

$$p(y)dy = \frac{1}{\sqrt{2\pi}} e^{-y^2/2} dy \quad (7.3.11)$$

Consider the transformation between two uniform deviates on (0,1), x_1, x_2 , and two quantities y_1, y_2 ,

$$\begin{aligned} y_1 &= \sqrt{-2 \ln x_1} \cos 2\pi x_2 \\ y_2 &= \sqrt{-2 \ln x_1} \sin 2\pi x_2 \end{aligned} \quad (7.3.12)$$

Equivalently we can write

$$\begin{aligned} x_1 &= \exp \left[-\frac{1}{2}(y_1^2 + y_2^2) \right] \\ x_2 &= \frac{1}{2\pi} \arctan \frac{y_2}{y_1} \end{aligned} \quad (7.3.13)$$

Now the Jacobian determinant can readily be calculated (try it!):

$$\frac{\partial(x_1, x_2)}{\partial(y_1, y_2)} = \begin{vmatrix} \frac{\partial x_1}{\partial y_1} & \frac{\partial x_1}{\partial y_2} \\ \frac{\partial x_2}{\partial y_1} & \frac{\partial x_2}{\partial y_2} \end{vmatrix} = - \left[\frac{1}{\sqrt{2\pi}} e^{-y_1^2/2} \right] \left[\frac{1}{\sqrt{2\pi}} e^{-y_2^2/2} \right] \quad (7.3.14)$$

Since this is the product of a function of y_2 alone and a function of y_1 alone, we see that each y is independently distributed according to the normal distribution (7.3.11).

One further trick is useful in applying (7.3.12). Suppose that, instead of picking uniform deviates x_1 and x_2 in the unit square, we instead pick v_1 and v_2 as the ordinate and abscissa of a random point inside the unit circle around the origin. Then the sum of their squares, $R^2 \equiv v_1^2 + v_2^2$, is a uniform deviate, which can be used for x_1 , while the angle that (v_1, v_2) defines with respect to the v_1 -axis can serve as the random angle $2\pi x_2$. What's the advantage? It's that the cosine and sine in (7.3.12) can now be written as $v_1/\sqrt{R^2}$ and $v_2/\sqrt{R^2}$, obviating the trigonometric function calls! (In the next section we will generalize this trick considerably.)

Code for generating normal deviates by the Box-Muller method follows. Consider it for pedagogical use only, because a significantly faster method for generating normal deviates is coming, below, in §7.3.9.

deviates.h

```

struct Normaldev_BM : Ran {
Structure for normal deviates.
    Doub mu,sig;
    Doub storedval;
    Normaldev_BM(Doub mmu, Doub ssig, Ullong i)
    : Ran(i), mu(mmu), sig(ssig), storedval(0.) {}
    Constructor arguments are  $\mu$ ,  $\sigma$ , and a random sequence seed.
    Doub dev() {
    Return a normal deviate.
        Doub v1,v2,rsq,fac;
        if (storedval == 0.) {
            do {
                v1=2.0*doub()-1.0;
                v2=2.0*doub()-1.0;
                rsq=v1*v1+v2*v2;
            } while (rsq >= 1.0 || rsq == 0.0);
            fac=sqrt(-2.0*log(rsq)/rsq);
            storedval = v1*fac;
            return mu + sig*v2*fac;
        } else {
            fac = storedval;
            storedval = 0.;
            return mu + sig*fac;
        }
    }
};

```

We don't have an extra deviate handy, so
pick two uniform numbers in the square extending from -1 to +1 in each direction, see if they are in the unit circle, or try again.
Now make the Box-Muller transformation to get two normal deviates. Return one and save the other for next time.
We have an extra deviate handy,
so return it.

7.3.5 Rayleigh Deviates

The *Rayleigh distribution* is defined for positive z by

$$p(z)dz = z \exp\left(-\frac{1}{2}z^2\right) dz \quad (z > 0) \quad (7.3.15)$$

Since the indefinite integral can be done analytically, and the result easily inverted, a simple transformation method from a uniform deviate x results:

$$z = \sqrt{-2 \ln x}, \quad x \sim U(0, 1) \quad (7.3.16)$$

A Rayleigh deviate z can also be generated from two normal deviates y_1 and y_2 by

$$z = \sqrt{y_1^2 + y_2^2}, \quad y_1, y_2 \sim N(0, 1) \quad (7.3.17)$$

Indeed, the relation between equations (7.3.16) and (7.3.17) is immediately evident in the equation for the Box-Muller method, equation (7.3.12), if we square and sum that method's two normal deviates y_1 and y_2 .

7.3.6 Rejection Method

The *rejection method* is a powerful, general technique for generating random deviates whose distribution function $p(x)dx$ (probability of a value occurring between x and $x + dx$) is known and computable. The rejection method does not require that the cumulative distribution function (indefinite integral of $p(x)$) be readily computable, much less the inverse of that function — which was required for the transformation method in the previous section.

The rejection method is based on a simple geometrical argument (Figure 7.3.2):

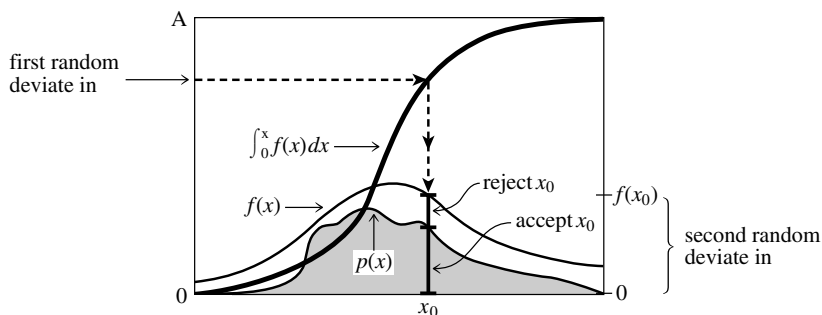


Figure 7.3.2. Rejection method for generating a random deviate x from a known probability distribution $p(x)$ that is everywhere less than some other function $f(x)$. The transformation method is first used to generate a random deviate x of the distribution f (compare Figure 7.3.1). A second uniform deviate is used to decide whether to accept or reject that x . If it is rejected, a new deviate of f is found, and so on. The ratio of accepted to rejected points is the ratio of the area under p to the area between p and f .

Draw a graph of the probability distribution $p(x)$ that you wish to generate, so that the area under the curve in any range of x corresponds to the desired probability of generating an x in that range. If we had some way of choosing a random point *in two dimensions*, with uniform probability in the *area* under your curve, then the x value of that random point would have the desired distribution.

Now, on the same graph, draw any other curve $f(x)$ that has finite (not infinite) area and lies everywhere *above* your original probability distribution. (This is always possible, because your original curve encloses only unit area, by definition of probability.) We will call this $f(x)$ the *comparison function*. Imagine now that you have some way of choosing a random point in two dimensions that is uniform in the area under the comparison function. Whenever that point lies outside the area under the original probability distribution, we will *reject* it and choose another random point. Whenever it lies inside the area under the original probability distribution, we will *accept* it.

It should be obvious that the accepted points are uniform in the accepted area, so that their x values have the desired distribution. It should also be obvious that the fraction of points rejected just depends on the ratio of the area of the comparison function to the area of the probability distribution function, not on the details of shape of either function. For example, a comparison function whose area is less than 2 will reject fewer than half the points, even if it approximates the probability function very badly at some values of x , e.g., remains finite in some region where $p(x)$ is zero.

It remains only to suggest how to choose a uniform random point in two dimensions under the comparison function $f(x)$. A variant of the transformation method (§7.3) does nicely: Be sure to have chosen a comparison function whose indefinite integral is known analytically, and is also analytically invertible to give x as a function of “area under the comparison function to the left of x .” Now pick a uniform deviate between 0 and A , where A is the total area under $f(x)$, and use it to get a corresponding x . Then pick a uniform deviate between 0 and $f(x)$ as the y value for the two-dimensional point. Finally, accept or reject according to whether it is respectively less than or greater than $p(x)$.

So, to summarize, the rejection method for some given $p(x)$ requires that one find, once and for all, some reasonably good comparison function $f(x)$. Thereafter,

each deviate generated requires two uniform random deviates, one evaluation of f (to get the coordinate y) and one evaluation of p (to decide whether to accept or reject the point x, y). Figure 7.3.1 illustrates the whole process. Then, of course, this process may need to be repeated, on the average, A times before the final deviate is obtained.

7.3.7 Cauchy Deviates

The “further trick” described following equation (7.3.14) in the context of the Box-Muller method is now seen to be a rejection method for getting trigonometric functions of a uniformly random angle. If we combine this with the explicit formula, equation (6.14.6), for the inverse cdf of the Cauchy distribution (see §6.14.2), we can generate Cauchy deviates quite efficiently.

```
struct Cauchydev : Ran {
Structure for Cauchy deviates.
    Doub mu,sig;
    Cauchydev(Doub mmu, Doub ssig, Ullong i) : Ran(i), mu(mmu), sig(ssig) {}
    Constructor arguments are  $\mu$ ,  $\sigma$ , and a random sequence seed.
    Doub dev() {
    Return a Cauchy deviate.
        Doub v1,v2;
        do {
            Find a random point in the unit semicircle.
            v1=2.0*doub()-1.0;
            v2=doub();
        } while (SQR(v1)+SQR(v2) >= 1. || v2 == 0.);
        return mu + sig*v1/v2;
        Ratio of its coordinates is the tangent of a
        random angle.
    }
};
```

deviates.h

7.3.8 Ratio-of-Uniforms Method

In finding Cauchy deviates, we took the ratio of two uniform deviates chosen to lie within the unit circle. If we generalize to shapes other than the unit circle, and combine it with the principle of the rejection method, a powerful variant emerges. Kinderman and Monahan [1] showed that deviates of virtually *any* probability distribution $p(x)$ can be generated by the following rather amazing prescription:

- Construct the region in the (u, v) plane bounded by $0 \leq u \leq [p(v/u)]^{1/2}$.
- Choose two deviates, u and v , that lie uniformly in this region.
- Return v/u as the deviate.

Proof: We can represent the ordinary rejection method by the equation in the (x, p) plane,

$$p(x)dx = \int_{p'=0}^{p'=p(x)} dp' dx \quad (7.3.18)$$

Since the integrand is 1, we are justified in sampling uniformly in (x, p') as long as p' is within the limits of the integral (that is, $0 < p' < p(x)$). Now make the change of variable

$$\begin{aligned} \frac{v}{u} &= x \\ u^2 &= p \end{aligned} \quad (7.3.19)$$

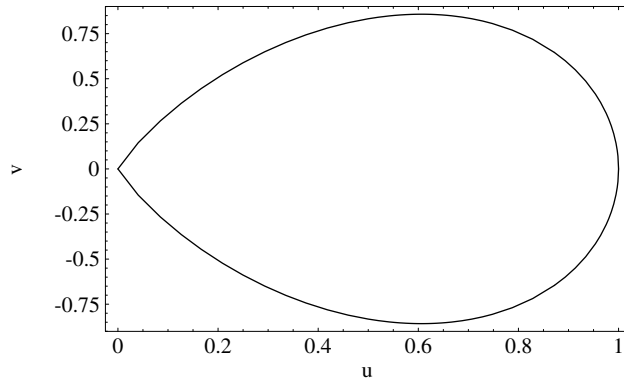


Figure 7.3.3. Ratio-of-uniforms method. The interior of this teardrop shape is the acceptance region for the normal distribution: If a random point is chosen inside this region, then the ratio v/u will be a normal deviate.

Then equation (7.3.18) becomes

$$p(x)dx = \int_{p'=0}^{p'=p(x)} dp' dx = \int_{u=0}^{u=\sqrt{p(x)}} \frac{\partial(p, x)}{\partial(u, v)} du dv = 2 \int_{u=0}^{u=\sqrt{p(v/u)}} du dv \quad (7.3.20)$$

because (as you can work out) the Jacobian determinant is the constant 2. Since the new integrand is constant, uniform sampling in (u, v) with the limits indicated for u is equivalent to the rejection method in (x, p) .

The above limits on u very often define a region that is “teardrop” shaped. To see why, note that the locii of constant $x = v/u$ are radial lines. Along each radial, the acceptance region goes from the origin to a point where $u^2 = p(x)$. Since most probability distributions go to zero for both large and small x , the acceptance region accordingly shrinks toward the origin along radials, producing a teardrop. Of course, it is the exact shape of this teardrop that matters. Figure 7.3.3 shows the shape of the acceptance region for the case of the normal distribution.

Typically this *ratio-of-uniforms* method is used when the desired region can be closely bounded by a rectangle, parallelogram, or some other shape that is easy to sample uniformly. Then, we go from sampling the easy shape to sampling the desired region by rejection of points outside the desired region.

An important adjunct to the ratio-of-uniforms method is the idea of a *squeeze*. A squeeze is any easy-to-compute shape that tightly bounds the region of acceptance of a rejection method, either from the inside or from the outside. Best of all is when you have squeezes on both sides. Then you can immediately reject points that are outside the outer squeeze and immediately accept points that are inside the inner squeeze. Only when you have the bad luck of drawing a point between the two squeezes do you actually have to do the more lengthy computation of comparing with the actual rejection boundary. Squeezes are useful both in the ordinary rejection method and in the ratio-of-uniforms method.

7.3.9 Normal Deviates by Ratio-of-Uniforms

Leva [2] has given an algorithm for normal deviates that uses the ratio-of-uniforms method with great success. He uses quadratic curves to provide both inner

and outer squeezes that hug the desired region in the (u, v) plane (Figure 7.3.3). Only about 1% of the time is it necessary to calculate an exact boundary (requiring a logarithm).

The resulting code looks so simple and “un-transcendental” that it may be hard to believe that exact normal deviates are generated. But they are!

```

struct Normaldev : Ran {
Structure for normal deviates.
    Doub mu,sig;
    Normaldev(Doub mmu, Doub ssig, Ullong i)
    : Ran(i), mu(mmu), sig(ssig){}
    Constructor arguments are  $\mu$ ,  $\sigma$ , and a random sequence seed.
    Doub dev() {
    Return a normal deviate.
        Doub u,v,x,y,q;
        do {
            u = doub();
            v = 1.7156*(doub()-0.5);
            x = u - 0.449871;
            y = abs(v) + 0.386595;
            q = SQR(x) + y*(0.19600*y-0.25472*x);
        } while (q > 0.27597
            && (q > 0.27846 || SQR(v) > -4.*log(u)*SQR(u)));
        return mu + sig*v/u;
    }
};

```

deviates.h

Note that the `while` clause makes use of C’s (and C++’s) guarantee that logical expressions are evaluated conditionally: If the first operand is sufficient to determine the outcome, the second is not evaluated at all. With these rules, the logarithm is evaluated only when q is between 0.27597 and 0.27846.

On average, each normal deviate uses 2.74 uniform deviates. By the way, even though the various constants are given only to six digits, the method is exact (to full double precision). Small perturbations of the bounding curves are of no consequence. The accuracy is implicit in the (rare) evaluations of the exact boundary.

7.3.10 Gamma Deviates

The distribution $\text{Gamma}(\alpha, \beta)$ was described in §6.14.9. The β parameter enters only as a scaling,

$$\text{Gamma}(\alpha, \beta) \cong \frac{1}{\beta} \text{Gamma}(\alpha, 1) \quad (7.3.21)$$

(Translation: To generate a $\text{Gamma}(\alpha, \beta)$ deviate, generate a $\text{Gamma}(\alpha, 1)$ deviate and divide it by β .)

If α is a small positive integer, a fast way to generate $x \sim \text{Gamma}(\alpha, 1)$ is to use the fact that it is distributed as the waiting time to the α th event in a Poisson random process of unit mean. Since the time between two consecutive events is just the exponential distribution $\text{Exponential}(1)$, you can simply add up α exponentially distributed waiting times, i.e., logarithms of uniform deviates. Even better, since the sum of logarithms is the logarithm of the product, you really only have to compute the product of α uniform deviates and then take the log. Because this is such a special case, however, we don’t include it in the code below.

When $\alpha < 1$, the gamma distribution's density function is not bounded, which is inconvenient. However, it turns out [4] that if

$$y \sim \text{Gamma}(\alpha + 1, 1), \quad u \sim \text{Uniform}(0, 1) \quad (7.3.22)$$

then

$$yu^{1/\alpha} \sim \text{Gamma}(\alpha, 1) \quad (7.3.23)$$

We will use this in the code below.

For $\alpha > 1$, Marsaglia and Tsang [5] give an elegant rejection method based on a simple transformation of the gamma distribution combined with a squeeze. After transformation, the gamma distribution can be bounded by a Gaussian curve whose area is never more than 5% greater than that of the gamma curve. The cost of a gamma deviate is thus only a little more than the cost of the normal deviate that is used to sample the comparison function. The following code gives the precise formulation; see the original paper for a full explanation.

```

deviates.h  struct Gammadev : Normaldev {
            Structure for gamma deviates.
            Doub alph, oalph, bet;
            Doub a1,a2;
            Gammadev(Doub aalph, Doub bbet, Ullong i)
            : Normaldev(0.,1.,i), alph(aalph), oalph(aalph), bet(bbet) {
            Constructor arguments are  $\alpha$ ,  $\beta$ , and a random sequence seed.
                if (alph <= 0.) throw("bad alph in Gammadev");
                if (alph < 1.) alph += 1.;
                a1 = alph-1./3.;
                a2 = 1./sqrt(9.*a1);
            }
            Doub dev() {
            Return a gamma deviate by the method of Marsaglia and Tsang.
                Doub u,v,x;
                do {
                    do {
                        x = Normaldev::dev();
                        v = 1. + a2*x;
                    } while (v <= 0.);
                    v = v*v*v;
                    u = doub();
                } while (u > 1. - 0.331*SQR(SQR(x)) &&
                        log(u) > 0.5*SQR(x) + a1*(1.-v+log(v))); Rarely evaluated.
                if (alph == oalph) return a1*v/bet;
                else {
                    Case where  $\alpha < 1$ , per Ripley.
                    do u=doub(); while (u == 0.);
                    return pow(u,1./oalph)*a1*v/bet;
                }
            }
        };

```

There exists a sum rule for gamma deviates. If we have a set of independent deviates y_i with possibly different α_i 's, but sharing a common value of β ,

$$y_i \sim \text{Gamma}(\alpha_i, \beta) \quad (7.3.24)$$

then their sum is also a gamma deviate,

$$y \equiv \sum_i y_i \sim \text{Gamma}(\alpha_T, \beta), \quad \alpha_T = \sum_i \alpha_i \quad (7.3.25)$$

If the α_i 's are integers, you can see how this relates to the discussion of Poisson waiting times above.

7.3.11 Distributions Easily Generated by Other Deviates

From normal, gamma and uniform deviates, we get a bunch of other distributions for free. Important: When you are going to combine their results, be sure that all distinct instances of `Normaldist`, `Gammadist`, and `Ran` have different random seeds! (`Ran` and its derived classes are sufficiently robust that seeds $i, i + 1, \dots$ are fine.)

Chi-Square Deviates (cf. §6.14.8)

This one is easy:

$$\text{Chisquare}(\nu) \cong \text{Gamma}\left(\frac{\nu}{2}, \frac{1}{2}\right) \cong 2 \text{Gamma}\left(\frac{\nu}{2}, 1\right) \quad (7.3.26)$$

Student-t Deviates (cf. §6.14.3)

Deviates from the Student-t distribution can be generated by a method very similar to the Box-Muller method. The analog of equation (7.3.12) is

$$y = \sqrt{\nu(u_1^{-2/\nu} - 1)} \cos 2\pi u_2 \quad (7.3.27)$$

If u_1 and u_2 are independently uniform, $U(0, 1)$, then

$$y \sim \text{Student}(\nu, 0, 1) \quad (7.3.28)$$

or

$$\mu + \sigma y \sim \text{Student}(\nu, \mu, \sigma) \quad (7.3.29)$$

Unfortunately, you can't do the Box-Muller trick of getting two deviates at a time, because the Jacobian determinant analogous to equation (7.3.14) does not factorize. You might want to use the polar method anyway, just to get $\cos 2\pi u_2$, but its advantage is now not so large.

An alternative method uses the quotients of normal and gamma deviates. If we have

$$x \sim N(0, 1), \quad y \sim \text{Gamma}\left(\frac{\nu}{2}, \frac{1}{2}\right) \quad (7.3.30)$$

then

$$x \sqrt{\nu/y} \sim \text{Student}(\nu, 0, 1) \quad (7.3.31)$$

Beta Deviates (cf. §6.14.11)

If

$$x \sim \text{Gamma}(\alpha, 1), \quad y \sim \text{Gamma}(\beta, 1) \quad (7.3.32)$$

then

$$\frac{x}{x + y} \sim \text{Beta}(\alpha, \beta) \quad (7.3.33)$$

F-Distribution Deviates (cf. §6.14.10)

If

$$x \sim \text{Beta}\left(\frac{1}{2}\nu_1, \frac{1}{2}\nu_2\right) \quad (7.3.34)$$

(see equation 7.3.33), then

$$\frac{\nu_2 x}{\nu_1(1 - x)} \sim F(\nu_1, \nu_2) \quad (7.3.35)$$

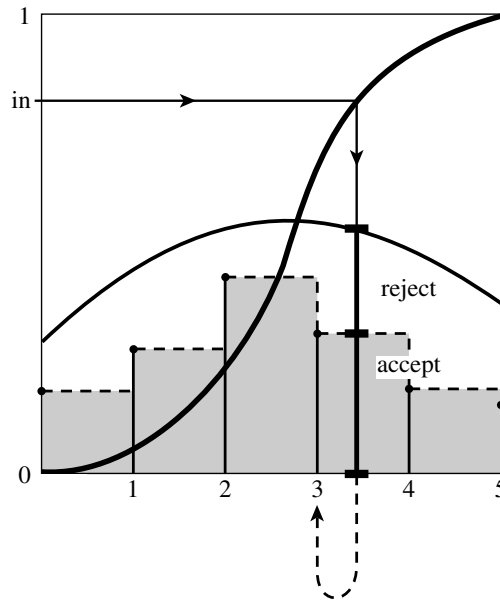


Figure 7.3.4. Rejection method as applied to an integer-valued distribution. The method is performed on the step function shown as a dashed line, yielding a real-valued deviate. This deviate is rounded down to the next lower integer, which is output.

7.3.12 Poisson Deviates

The Poisson distribution, $\text{Poisson}(\lambda)$, previously discussed in §6.14.13, is a discrete distribution, so its deviates will be integers, k . To use the methods already discussed, it is convenient to convert the Poisson distribution into a continuous distribution by the following trick: Consider the finite probability $p(k)$ as being spread out uniformly into the interval from k to $k + 1$. This defines a continuous distribution $q_\lambda(k)dk$ given by

$$q_\lambda(k)dk = \frac{\lambda^{\lfloor k \rfloor} e^{-\lambda}}{\lfloor k \rfloor!} dk \quad (7.3.36)$$

where $\lfloor k \rfloor$ represents the largest integer $\leq k$. If we now use a rejection method, or any other method, to generate a (noninteger) deviate from (7.3.36), and then take the integer part of that deviate, it will be as if drawn from the discrete Poisson distribution. (See Figure 7.3.4.) This trick is general for any integer-valued probability distribution. Instead of the “floor” operator, one can equally well use “ceiling” or “nearest” — anything that spreads the probability over a unit interval.

For λ large enough, the distribution (7.3.36) is qualitatively bell-shaped (albeit with a bell made out of small, square steps). In that case, the ratio-of-uniforms method works well. It is not hard to find simple inner and outer squeezes in the (u, v) plane of the form $v^2 = Q(u)$, where $Q(u)$ is a simple polynomial in u . The only trick is to allow a big enough gap between the squeezes to enclose the true, jagged, boundaries for all values of λ . (Look ahead to Figure 7.3.5 for a similar example.)

For intermediate values of λ , the jaggedness is so large as to render squeezes impractical, but the ratio-of-uniforms method, unadorned, still works pretty well.

For small λ , we can use an idea similar to that mentioned above for the gamma distribution in the case of integer a . When the sum of independent exponential

deviates first exceeds λ , their number (less 1) is a Poisson deviate k . Also, as explained for the gamma distribution, we can multiply uniform deviates from $U(0, 1)$ instead of adding deviates from Exponential (1).

These ideas produce the following routine.

```

struct Poissondev : Ran {
Structure for Poisson deviates.
    Doub lambda, sqlam, loglam, lamexp, lambold;
    VecDoub logfact;
    Int swch;
    Poissondev(Doub llambda, Ullong i) : Ran(i), lambda(llambda),
        logfact(1024,-1.), lambold(-1.) {}
Constructor arguments are  $\lambda$  and a random sequence seed.
    Int dev() {
Return a Poisson deviate using the most recently set value of  $\lambda$ .
        Doub u,u2,v,v2,p,t,lfac;
        Int k;
        if (lambda < 5.) {
            Will use product of uniforms method.
            if (lambda != lambold) lamexp=exp(-lambda);
            k = -1;
            t=1.;
            do {
                ++k;
                t *= doub();
            } while (t > lamexp);
        } else {
            Will use ratio-of-uniforms method.
            if (lambda != lambold) {
                sqlam = sqrt(lambda);
                loglam = log(lambda);
            }
            for (;;) {
                u = 0.64*doub();
                v = -0.68 + 1.28*doub();
                if (lambda > 13.5) {
                    Outer squeeze for fast rejection.
                    v2 = SQR(v);
                    if (v >= 0.) {if (v2 > 6.5*u*(0.64-u)*(u+0.2)) continue;}
                    else {if (v2 > 9.6*u*(0.66-u)*(u+0.07)) continue;}
                }
                k = Int(floor(sqlam*(v/u)+lambda+0.5));
                if (k < 0) continue;
                u2 = SQR(u);
                if (lambda > 13.5) {
                    Inner squeeze for fast acceptance.
                    if (v >= 0.) {if (v2 < 15.2*u2*(0.61-u)*(0.8-u)) break;}
                    else {if (v2 < 6.76*u2*(0.62-u)*(1.4-u)) break;}
                }
                if (k < 1024) {
                    if (logfact[k] < 0.) logfact[k] = gammln(k+1.);
                    lfac = logfact[k];
                } else lfac = gammln(k+1.);
                p = sqlam*exp(-lambda + k*loglam - lfac);
                if (u2 < p) break;
            }
        }
        lambold = lambda;
        return k;
    }
    Int dev(Doub llambda) {
Reset  $\lambda$  and then return a Poisson deviate.
        lambda = llambda;
        return dev();
    }
};

```

deviates.h

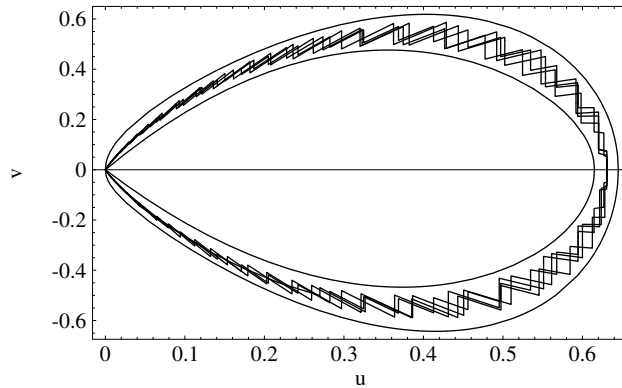


Figure 7.3.5. Ratio-of-uniforms method as applied to the generation of binomial deviates. Points are chosen randomly in the (u, v) -plane. The smooth curves are inner and outer squeezes. The jagged curves correspond to various binomial distributions with $n > 64$ and $np > 30$. An evaluation of the binomial probability is required only when the random point falls between the smooth curves.

In the regime $\lambda > 13.5$, the above code uses about 3.3 uniform deviates per output Poisson deviate and does 0.4 evaluations of the exact probability (costing an exponential and, for large k , a call to `gammln`).

`Poissondev` is slightly faster if you draw many deviates with the same value λ , using the `dev` function with no arguments, than if you vary λ on each call, using the one-argument overloaded form of `dev` (which is provided for just that purpose). The difference is just an extra exponential ($\lambda < 5$) or square root and logarithm ($\lambda \geq 5$). Note also the object's table of previously computed log-factorials. If your λ 's are as large as $\sim 10^3$, you might want to make the table larger.

7.3.13 Binomial Deviates

The generation of binomial deviates $k \sim \text{Binomial}(n, p)$ involves many of the same ideas as for Poisson deviates. The distribution is again integer-valued, so we use the same trick to convert it into a stepped continuous distribution. We can always restrict attention to the case $p \leq 0.5$, since the distribution's symmetries let us trivially recover the case $p > 0.5$.

When $n > 64$ and $np > 30$, we use the ratio-of-uniforms method, with squeezes shown in Figure 7.3.5. The cost is about 3.2 uniform deviates, plus 0.4 evaluations of the exact probability, per binomial deviate.

It would be foolish to waste much thought on the case where $n > 64$ and $np < 30$, because it is so easy simply to tabulate the cdf, say for $0 \leq k < 64$, and then loop over k 's until the right one is found. (A bisection search, implemented below, is even better.) With a cdf table of length 64, the neglected probability at the end of the table is never larger than $\sim 10^{-20}$. (At 10^9 deviates per second, you could run 3000 years before losing a deviate.)

What is left is the interesting case $n < 64$, which we will explore in some detail, because it demonstrates the important concept of *bit-parallel random comparison*.

Analogous to the methods for gamma deviates with small integer a and for Poisson deviates with small λ , is this direct method for binomial deviates: Generate n uniform deviates in $U(0, 1)$. Count the number of them $< p$. Return the count as

$k \sim \text{Binomial}(n, p)$. Indeed this is essentially the definition of a binomial process!

The problem with the direct method is that it seems to require n uniform deviates, even when the mean value of k is much smaller. Would you be surprised if we told you that for $n \leq 64$ you can achieve the same goal with at most *seven* 64-bit uniform deviates, on average? Here is how.

Expand $p < 1$ into its first 5 bits, plus a residual,

$$p = b_1 2^{-1} + b_2 2^{-2} + \cdots + b_5 2^{-5} + p_r 2^{-5} \quad (7.3.37)$$

where each b_i is 0 or 1, and $0 \leq p_r \leq 1$.

Now imagine that you have generated and stored 64 uniform $U(0, 1)$ deviates, and that the 64-bit word P displays just the first bit of each of the 64. Compare each bit of P to b_1 . If the bits are the same, then we don't yet know whether that uniform deviate is less than or greater than p . But if the bits are *different*, then we know that the generator is less than p (in the case that $b_1 = 1$) or greater than p (in the case that $b_1 = 0$). If we keep a mask of "known" versus "unknown" cases, we can do these comparisons in a bit-parallel manner by bitwise logical operations (see code below to learn how). Now move on to the second bit, b_2 , in the same way. At each stage we change half the remaining unknowns to knowns. After five stages (for $n = 64$) there will be two remaining unknowns, on average, each of which we finish off by generating a new uniform and comparing it to p_r . (This requires a loop through the 64 bits; but since C++ has no bitwise "popcount" operation, we are stuck doing such a loop anyway. If you can do popcounts, you may be better off just doing more stages until the unknowns mask is zero.)

The trick is that the bits used in the five stages are not actually the leading five bits of 64 generators, they are just five independent 64-bit random integers. The number five was chosen because it minimizes $64 \times 2^{-j} + j$, the expected number of deviates needed.

So, the code for binomial deviates ends up with three separate methods: bit-parallel direct, cdf lookup (by bisection), and squeezed ratio-of-uniforms.

```

struct Binomialdev : Ran {
Structure for binomial deviates.
    Doub pp,p,pb,expnp,np,glnp,plog,pclog,sq;
    Int n,swch;
    Ullong uz,uo,unfin,diff,rltp;
    Int pbits[5];
    Doub cdf[64];
    Doub logfact[1024];
    Binomialdev(Int nn, Doub ppp, Ullong i) : Ran(i), pp(ppp), n(nn) {
Constructor arguments are n, p, and a random sequence seed.
        Int j;
        pb = p = (pp <= 0.5 ? pp : 1.0-pp);
        if (n <= 64) {
            uz=0;
            uo=0xffffffffffffffffLL;
            rltp = 0;
            for (j=0;j<5;j++) pbits[j] = 1 & ((Int)(pb * 2.));
            pb -= floor(pb);
            swch = 0;
        } else if (n*p < 30.) {
            cdf[0] = exp(n*log(1-p));
            for (j=1;j<64;j++) cdf[j] = cdf[j-1] + exp(gammln(n+1.)
                -gammln(j+1.)-gammln(n-j+1.)+j*log(p)+(n-j)*log(1.-p));
            swch = 1;

```

deviates.h

Will use bit-parallel direct method.

Leading bits of p (above) and remaining fraction.

Will use precomputed cdf table.

```

    } else {
        np = n*p;
        glnp=gammln(n+1.);
        plog=log(p);
        pclog=log(1.-p);
        sq=sqrt(np*(1.-p));
        if (n < 1024) for (j=0;j<=n;j++) logfact[j] = gammln(j+1.);
        swch = 2;
    }
}

Int dev() {
    Return a binomial deviate.
    Int j,k,kl,km;
    Doub y,u,v,u2,v2,b;
    if (swch == 0) {
        unfin = uo;
        for (j=0;j<5;j++) {
            diff = unfin & (int64()^(pbits[j]? uo : uz)); // Mask of diff.
            if (pbits[j]) rltp |= diff; // Set bits to 1, meaning ran < p.
            else rltp = rltp & ~diff; // Set bits to 0, meaning ran > p.
            unfin = unfin & ~diff; // Update unfinished status.
        }
        k=0;
        for (j=0;j<n;j++) {
            if (unfin & 1) {if (doub() < pb) ++k;} // Now we just count the events.
            else {if (rltp & 1) ++k;} // Clean up unresolved cases,
            unfin >>= 1; // or use bit answer.
            rltp >>= 1;
        }
    } else if (swch == 1) {
        Use stored cdf.
        y = doub();
        kl = -1;
        k = 64;
        while (k-kl>1) {
            km = (kl+k)/2;
            if (y < cdf[km]) k = km;
            else kl = km;
        }
    } else {
        Use ratio-of-uniforms method.
        for (;;) {
            u = 0.645*doub();
            v = -0.63 + 1.25*doub();
            v2 = SQR(v);
            Try squeeze for fast rejection:
            if (v >= 0.) {if (v2 > 6.5*u*(0.645-u)*(u+0.2)) continue;}
            else {if (v2 > 8.4*u*(0.645-u)*(u+0.1)) continue;}
            k = Int(floor(sq*(v/u)+np+0.5));
            if (k < 0) continue;
            u2 = SQR(u);
            Try squeeze for fast acceptance:
            if (v >= 0.) {if (v2 < 12.25*u2*(0.615-u)*(0.92-u)) break;}
            else {if (v2 < 7.84*u2*(0.615-u)*(1.2-u)) break;}
            b = sq*exp(glnp+k*plog+(n-k)*pclog // Only when we must.
                - (n < 1024 ? logfact[k]+logfact[n-k]
                  : gammln(k+1.)+gammln(n-k+1.)));
            if (u2 < b) break;
        }
    }
    if (p != pp) k = n - k;
    return k;
}
};

```


If you are in a situation where you are drawing only one or a few deviates each for many different values of n and/or p , you'll need to restructure the code so that n and p can be changed without creating a new instance of the object and without reinitializing the underlying Ran generator.

7.3.14 When You Need Greater Speed

In particular situations you can cut some corners to gain greater speed. Here are some suggestions.

- All of the algorithms in this section can be speeded up significantly by using `Ranq1` in §7.1 instead of `Ran`. We know of no reason not to do this. You can gain some further speed by coding `Ranq1`'s algorithm inline, thus eliminating the function calls.
- If you are using `Poissondev` or `Binomialdev` with large values of λ or n , then the above codes revert to calling `gammln`, which is slow. You can instead increase the length of the stored tables.
- For Poisson deviates with $\lambda < 20$, you may want to use a stored table of cdfs combined with bisection to find the value of k . The code in `Binomialdev` shows how to do this.
- If your need is for binomial deviates with small n , you can easily modify the code in `Binomialdev` to get multiple deviates ($\sim 64/n$, in fact) from each execution of the bit-parallel code.
- Do you need exact deviates, or would an approximation do? If your distribution of interest can be approximated by a normal distribution, consider substituting `Normaldev`, above, especially if you also code the uniform random generation inline.
- If you sum exactly 12 uniform deviates $U(0, 1)$ and then subtract 6, you get a pretty good approximation of a normal deviate $N(0, 1)$. This is definitely slower than `Normaldev` (not to mention less accurate) on a general-purpose CPU. However, there are reported to be some special-purpose signal processing chips in which all the operations can be done with integer arithmetic and in parallel.

See Gentle [3], Ripley [4], Devroye [6], Bratley [7], and Knuth [8] for many additional algorithms.

CITED REFERENCES AND FURTHER READING:

- Kinderman, A.J. and Monahan, J.F. 1977, "Computer Generation of Random Variables Using the Ratio of Uniform Deviates," *ACM Transactions on Mathematical Software*, vol. 3, pp. 257–260.[1]
- Leva, J.L. 1992. "A Fast Normal Random Number Generator," *ACM Transactions on Mathematical Software*, vol. 18, no. 4, pp. 449–453.[2]
- Gentle, J.E. 2003, *Random Number Generation and Monte Carlo Methods*, 2nd ed. (New York: Springer), Chapters 4–5.[3]
- Ripley, B.D. 1987, *Stochastic Simulation* (New York: Wiley).[4]
- Marsaglia, G. and Tsang W-W. 2000, "A Simple Method for Generating Gamma Variables," *ACM Transactions on Mathematical Software*, vol. 26, no. 3, pp. 363–372.[5]
- Devroye, L. 1986, *Non-Uniform Random Variate Generation* (New York: Springer).[6]